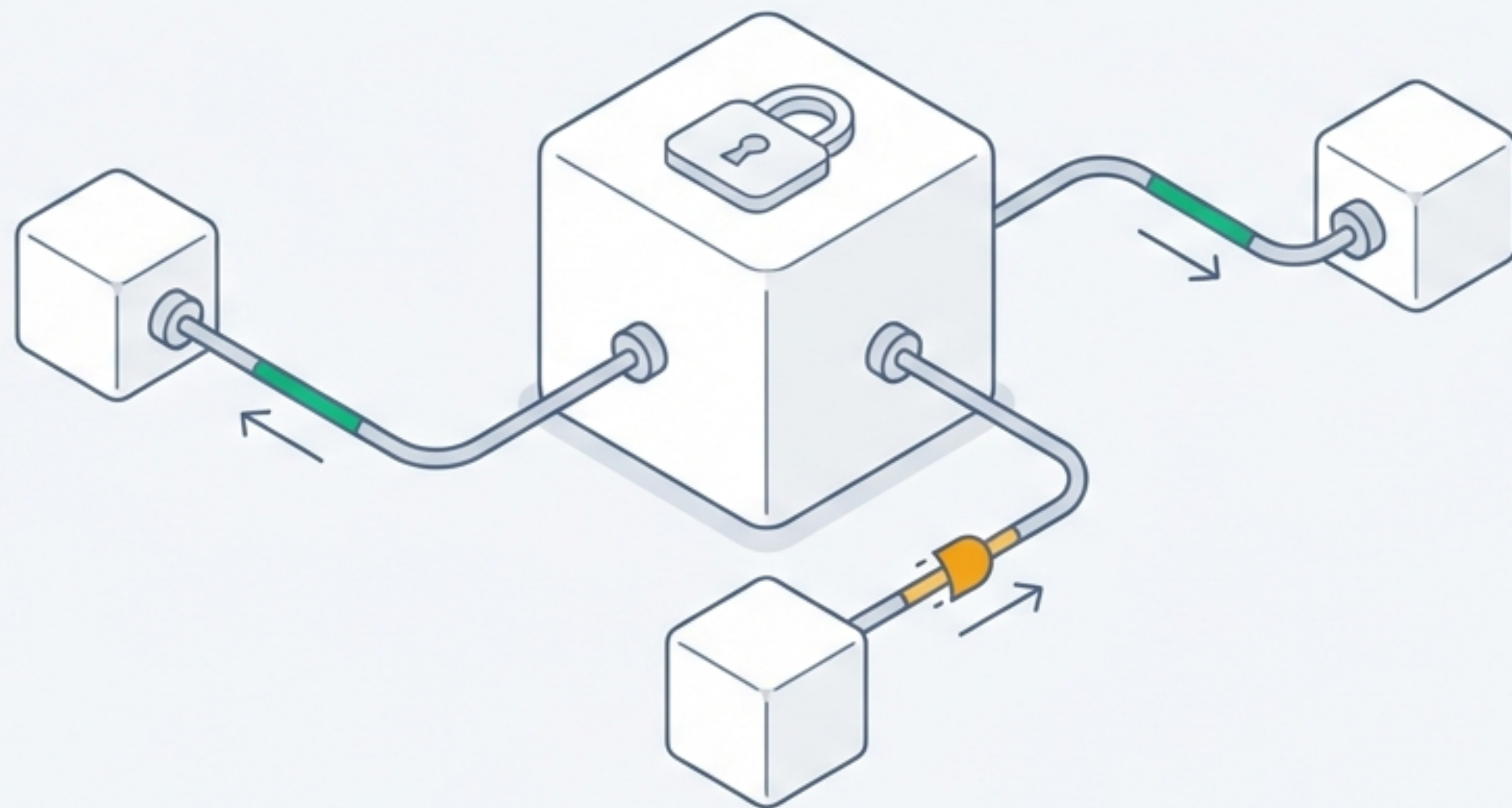


Noto Sans TC

Auth Center 中央認證系統技術分享

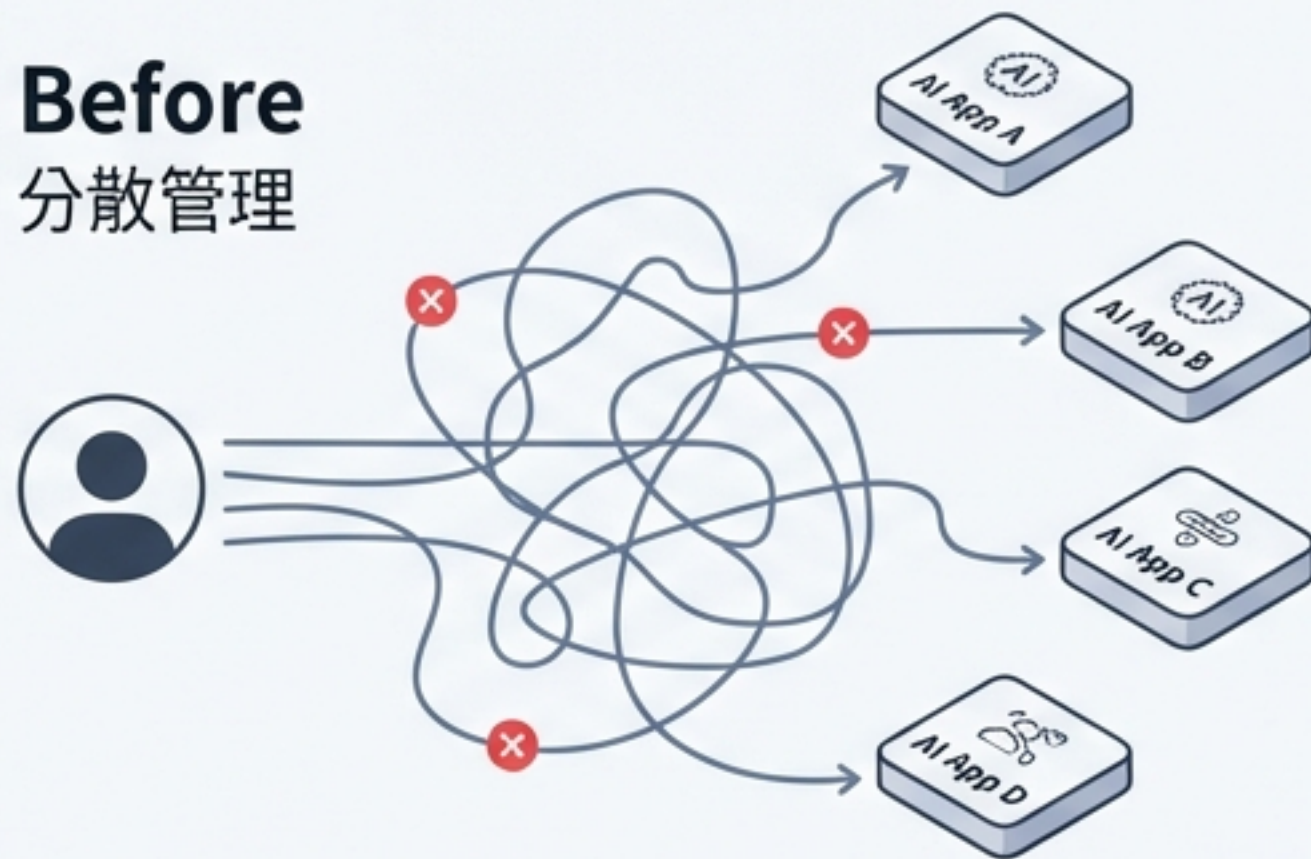


Noto Sans TC
統一 SSO 解決方案與 AI App 接入指南

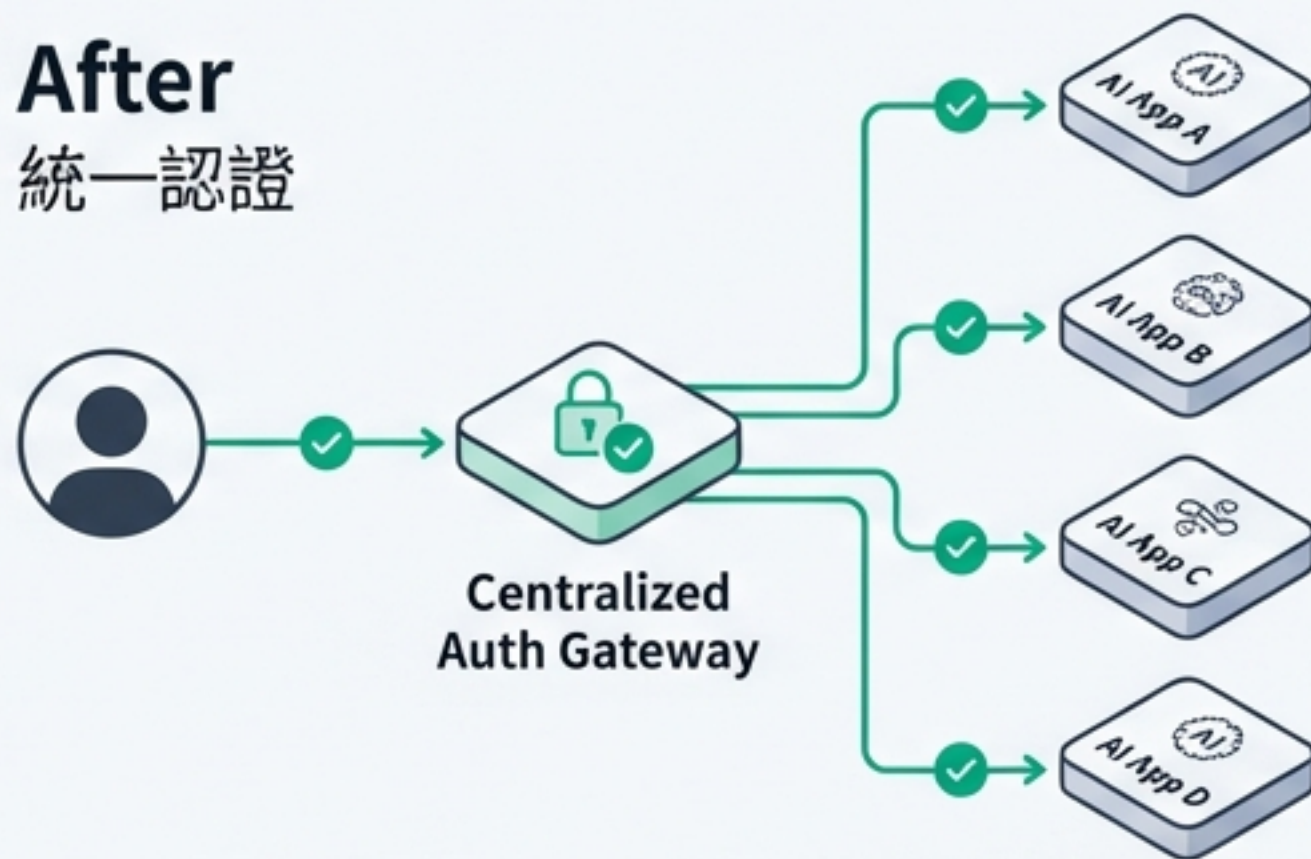
系統簡介：統一認證與權限管理

- ✓ **定位**：為內部 AI App 提供統一的單一登入 (SSO) 服務
- ✓ **痛點解決**：解決帳號分散，員工一組帳號存取所有 App
- ✓ **核心技術**：OAuth2 Authorization Code Flow + RS256 非對稱加密
- ✓ **權限模型**：Per-User-Per-App 細緻控管 (Read/Write/Admin)
- ✓ **雙層管理**：Super Admin (全局) 與 App Admin (個別應用)

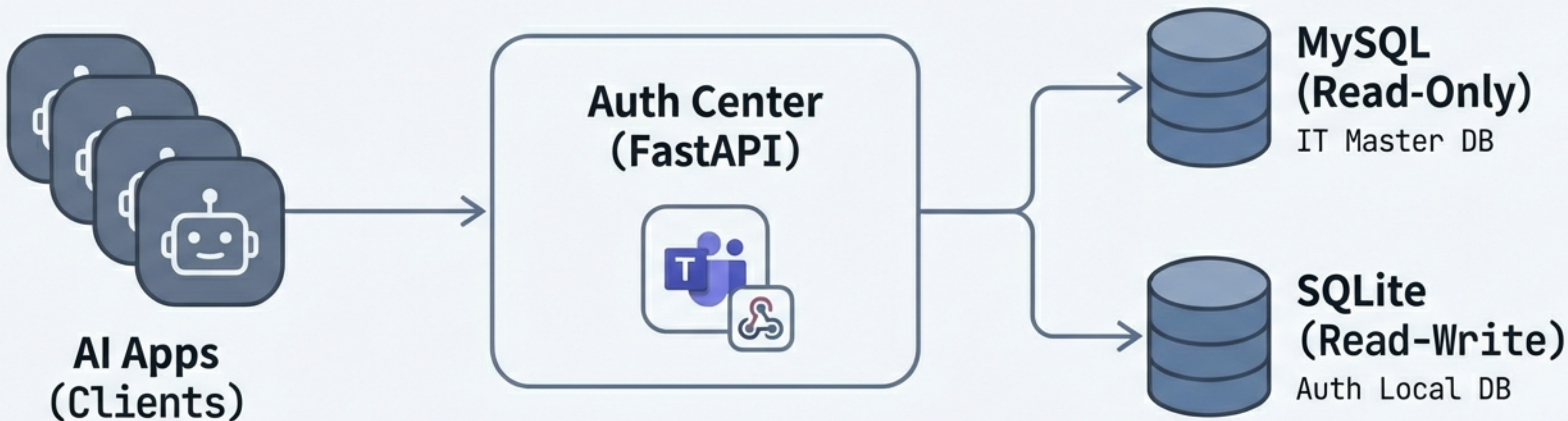
Before
分散管理



After
統一認證



系統架構總覽 (System Architecture)



Backend Framework:

FastAPI (Python 3.11+)

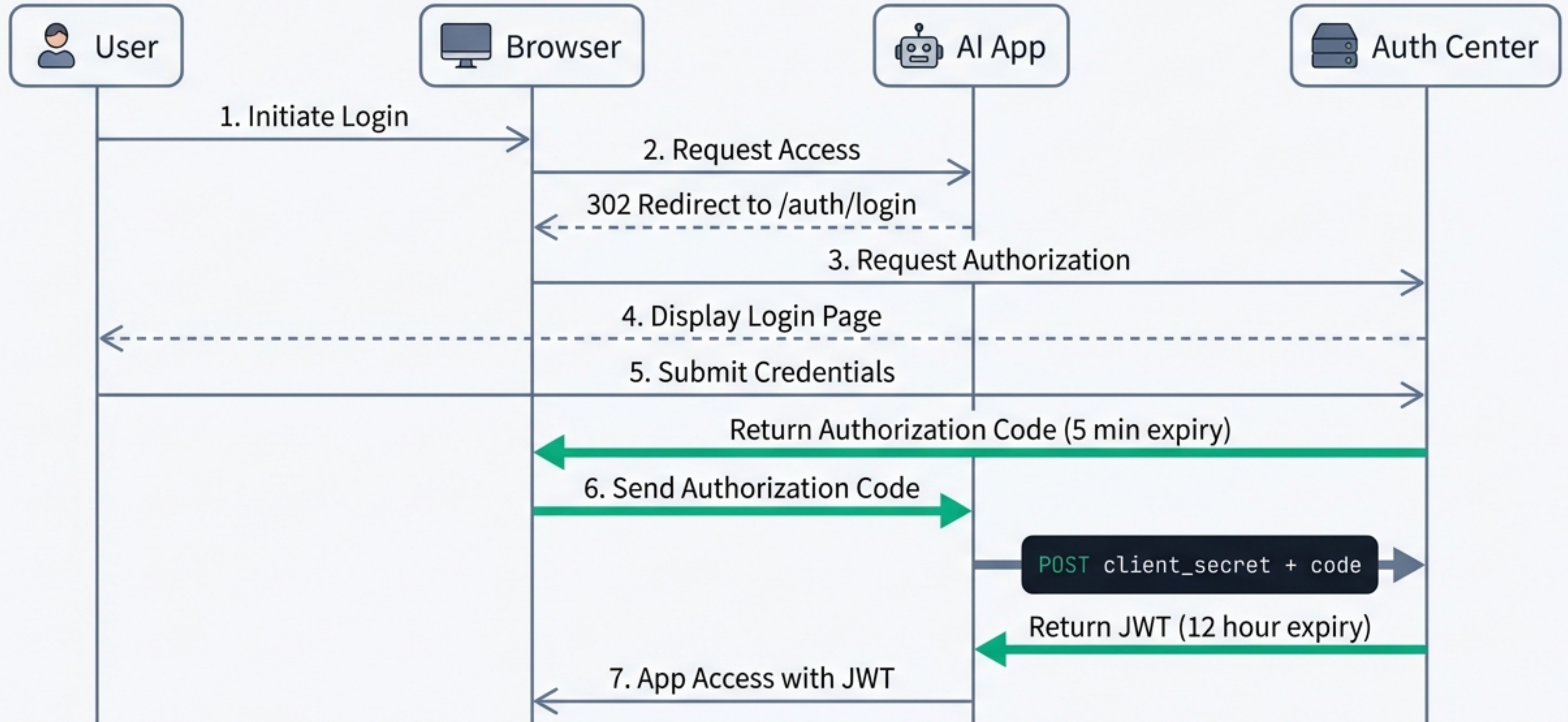
Security:

PyJWT (RS256) + bcrypt

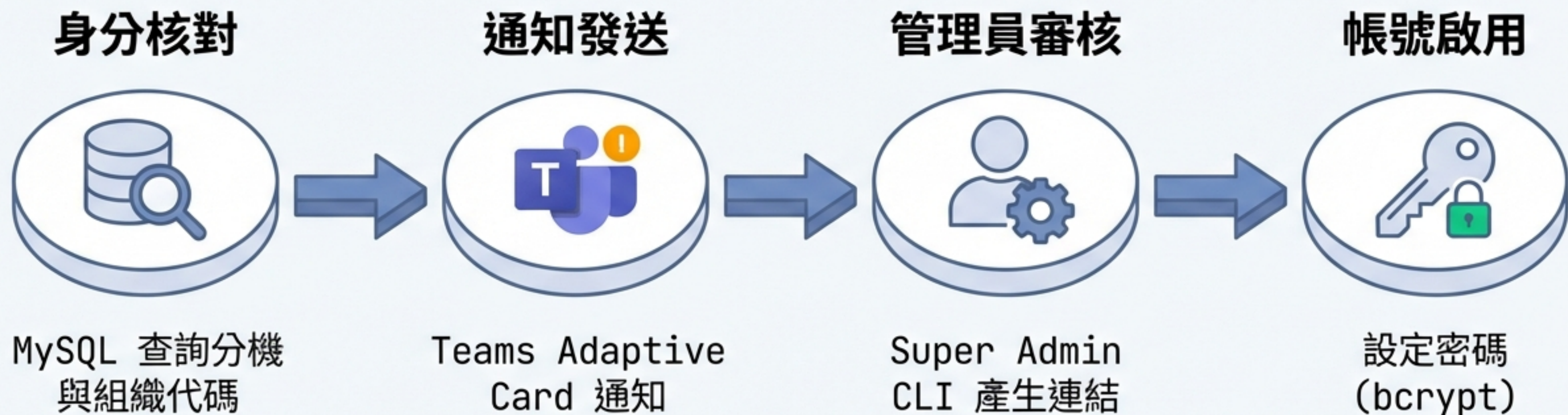
Config:

apps.yaml 註冊管理

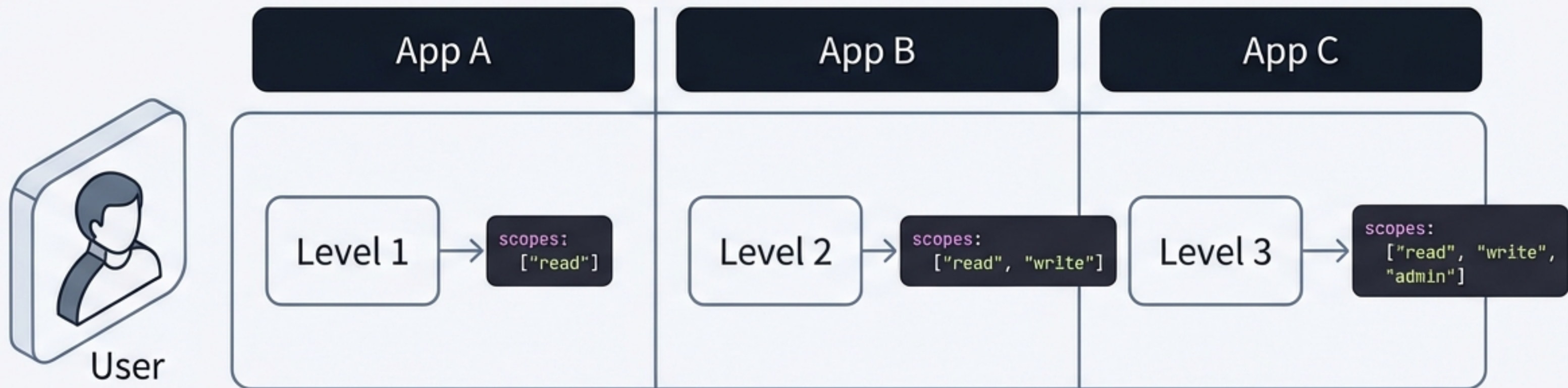
OAuth2 Authorization Code Flow



首次登入與身份驗證流程



Per-User-Per-App 權限模型



⚠ Global Filter

`allowed\_orgs` 白名單過濾 (e.g., IT, RD Only)

JWT Token 格式與安全 (RS256)

```
{  
  "alg": "RS256",  
  "sub": "kane.beh",  
  "org_id": "IT",  
  "aud": "ai_chat_app",  
  "scopes": ["read", "write"],  
  "exp": 1709043200  
}
```

🔒 RS256: Auth Center 私鑰簽名，App 公鑰驗證

👤 員工帳號

⚠️ **Critical Security:** 確保 Token 專屬於特定 App

⚙️ 權限範圍 (由 Level 映射)

🕒 有效期 12 小時

兩層 Admin 管理機制



Audit Log: 所有 Grant / Revoke / Login 操作皆寫入資料庫稽核

AI App 接入指南 (1): Onboarding Checklist

1

產生 Secret

使用 JetBrains Mono `bcrypt` 產生 `client_secret` hash



2

註冊 App

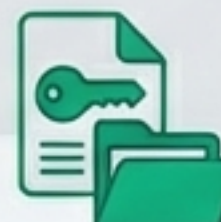
設定 JetBrains Mono `apps.yaml` (`app_id`, `redirect_uri`)



3

部署公鑰

將 JetBrains Mono `public.pem` 放入 App 專案目錄



4

環境變數

設定 JetBrains Mono `.env` (`AUTH_CENTER_URL`, `CLIENT_SECRET`)



注意：

**JetBrains
接入Brains
Mono**

`redirect_uri`
必須與設定檔
完全匹配
(Strict Match)

AI App 接入指南 (2): Callback 實作

Server-to-Server
Exchange

```
@app.get("/auth/callback")
async def auth_callback(code: str):
    # 1. Exchange Code for Token
    async with httpx.AsyncClient() as client:
        resp = await client.post(f"{AUTH_CENTER}/auth/token", json={
            "code": code,
            "client_secret": CLIENT_SECRET
        })

    # 2. Save to Secure Cookie
    response.set_cookie(key="access_token", value=token, httponly=True)
    return response
```

HttpOnly + Secure
(防止 XSS)

AI App 接入指南 (3): 權限驗證

```
def get_current_user(token: str = Cookie()):  
    # 1. Local Verification (No API Call)  
    payload = jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"], audience=APP_ID)  
    return payload
```

使用 Public Key 驗證簽名

驗證 aud 欄位 (Security Critical)

```
@app.get("/admin")  
def admin_panel(user = Depends(require_scopes(["admin"]))):  
    return {"msg": "Admin Access Granted"}
```

Scope 保護裝飾器

系統安全機制 (Security Mechanisms)



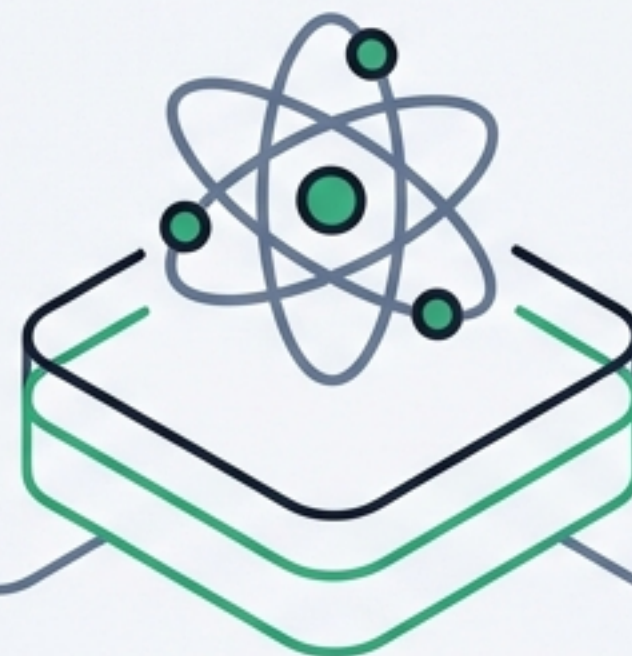
加密儲存

密碼 bcrypt 雜湊，
Token RS256 簽名



防禦攻擊

Rate Limiting
(10次/5分)，
防時序攻擊



Code 安全

Authorization Code
原子消耗 (Atomic)，
防 Replay



Admin 安全

管理員 Token
短效期 (2 hr)，
強制綁定 is_super

運維與 CLI 管理工具

```
user@host:~$ python scripts/generate_register_link.py
```

處理註冊請求

```
user@host:~$ python scripts/reset_password.py
```

應對忘記密碼

```
user@host:~$ python scripts/manage_permissions.py
```

權限授予/撤銷

適用情境：無法存取 Web UI 或需批量處理時

總結與資源

- 統一：解決帳號分散，提升體驗
- 安全：標準 OAuth2 + RSA 加密
- 彈性：Per-App 權限模型
- 資源：參考 `example_app/main.py`
獲取完整實作



Internal Repo / Docs